SYSTEM PROGRAMMING WITH PYTHON

13-Session University Teaching Plan

Operating Systems • Concurrency • Networking • Security • API Development

Course description

A practical course that introduces students to operating-system interfaces, Windows Registry access, socket programming, ports, multithreading, defensive security, REST APIs, and system automation using Python.

Suggested session length: 90–120 minutes

Recommended teaching balance: approximately 40% concepts and 60% demonstrations and exercises

Course outline at a glance

Session	Topic	Practical outcome
1	Introduction to system programming	Inspect processes, files, environment, and OS information
2	Windows Registry	Read selected registry keys safely
3	Sockets and network fundamentals	Explain client-server communication
4	TCP and UDP socket programming	Build a client and server
5	Ports and network services	Check a local service's reachability
6	Threads and multithreading	Run I/O-bound tasks concurrently
7	Synchronization and multithreaded servers	Protect shared state and serve multiple clients
8	Viruses, malware, and antivirus	Perform defensive file-integrity checks
9	APIs, HTTP, and REST	Consume an HTTP API and parse JSON
10	Building APIs with FastAPI	Implement CRUD endpoints
11	Programming languages for system programming	Compare Python with lower-level languages
12	System-related functions and automation	Inspect processes and run subprocesses safely
13	Integration project and review	Present a concurrent system-monitoring API

Detailed session plans

Session 1 — Introduction to System Programming

Objective: Understand how application software interacts with the operating system.

What to teach

- System programming vs. application programming.
- User space, kernel space, system calls, and OS APIs.
- Processes, threads, memory, files, and I/O.
- Python's role and limitations compared with C/C++.
- Inspecting OS information and process identifiers.

Practical demonstration

Demonstrate Python's `os` and `platform` modules: print the OS name/version, Python version, process ID, parent process ID, current directory, and selected environment variables.

Student exercise: Write a system-information script that reports the operating system, Python version, process ID, working directory, and selected environment variables.

Resources: Python os; platform.

Session 2 — Windows Registry

Objective: Understand the Registry structure and safely inspect application and OS configuration.

What to teach

- Registry keys, subkeys, values, and data types.
- Common hives: HKEY_CURRENT_USER, HKEY_LOCAL_MACHINE, HKEY_CLASSES_ROOT, HKEY_USERS, HKEY_CURRENT_CONFIG.
- Reading and writing values using winreg.
- Permissions, administrator privileges, backup, and troubleshooting.
- Safe modification of test keys only.

Practical demonstration

Read the Windows product name from a known registry key using Python's `winreg` module.

Student exercise: Read Windows version information and create then remove a test key under the current user's registry hive. Do not modify critical OS settings.

Resources: Python winreg documentation.

Session 3 — Sockets and Network Fundamentals

Objective: Understand how programs communicate over a network.

What to teach

- Client-server architecture and network protocol basics.
- IPv4/IPv6 addresses, hostnames, and ports.
- TCP/IP protocol stack; TCP vs. UDP.
- What a socket represents and socket endpoints.
- Blocking vs. non-blocking operations.

Practical demonstration

Use `socket.gethostname()` and `socket.gethostbyname()` to inspect a hostname and resolve a domain.

Student exercise: Resolve a domain name to an IP address and draw the steps from a client socket to a listening server.

Resources: Socket Programming HOWTO.

Session 4 — TCP and UDP Socket Programming

Objective: Build working network applications with Python sockets.

What to teach

- TCP socket lifecycle: socket, bind, listen, accept, connect.
- Sending and receiving bytes; sendall() and recv().
- Message boundaries and application-level framing.
- Connection closure, exceptions, and timeouts.
- UDP sendto() and recvfrom(); compare reliability and ordering.

Practical demonstration

Build a local TCP echo server and a separate client. Then demonstrate a simple UDP request/response pair.

Student exercise: Extend the echo server to support TIME, UPPERCASE, and EXIT commands; handle disconnects and malformed input.

Resources: Python socket documentation.

Session 5 — Ports and Network Services

Objective: Explain how network services use ports and how to diagnose reachability.

What to teach

- Well-known, registered, and dynamic/private port ranges.
- Common services: HTTP, HTTPS, DNS, and SSH.
- Listening ports vs. established connections.
- Address-in-use errors, firewalls, and permissions.
- Timeouts and basic troubleshooting; a failed connection is not proof that a port is closed.

Practical demonstration

Write a short client that attempts to connect to 127.0.0.1 on a chosen port with a timeout.

Student exercise: Run the TCP server from Session 4 and check its reachability before, during, and after it is running. Explain possible causes of failure.

Resources: Python socket documentation; Wireshark documentation.

Session 6 — Threads and Multithreading

Objective: Understand concurrency and implement multithreaded Python programs.

What to teach

- Process vs. thread; concurrency vs. parallelism.
- threading.Thread, start(), join(), and thread arguments.
- Daemon and non-daemon threads.
- Shared state and race conditions.
- I/O-bound vs. CPU-bound work and the GIL.

Practical demonstration

Run three worker threads that simulate I/O using time.sleep(), then compare their execution with sequential execution.

Student exercise: Measure sequential vs. threaded execution with time.perf_counter() and explain why the result differs for I/O-bound and CPU-bound tasks.

Resources: Python threading documentation.

Session 7 — Synchronization and Multithreaded Servers

Objective: Coordinate concurrent tasks and safely serve multiple clients.

What to teach

- Critical sections, race conditions, and Lock/RLock.
- Semaphore, Event, and queue.Queue.
- Deadlocks and prevention strategies.
- Thread pools with concurrent.futures.
- One thread per client vs. bounded worker pools; errors and shutdown.

Practical demonstration

Use a lock to protect a shared counter, then modify the TCP server to handle multiple clients concurrently.

Student exercise: Run multiple clients simultaneously, verify that each receives a response, and explain how locks protect shared state.

Resources: concurrent.futures; queue.

Session 8 — Viruses, Malware, and Antivirus

Objective: Understand malware categories and practice defensive system security.

What to teach

- Virus, worm, Trojan, ransomware, spyware, and rootkit.
- Common infection and propagation mechanisms.
- Signature, heuristic, and behavior-based detection.
- File hashes and integrity checking.
- Safe handling, least privilege, updates, backups, incident response, and ethics.

Practical demonstration

Demonstrate a SHA-256 file hash using hashlib and explain how changing a file changes its digest.

Student exercise: Build a file-integrity checker for a dedicated test directory. Record hashes and report changed or missing files; use benign files only.

Resources: NIST Cybersecurity resources; hashlib documentation.

Session 9 — APIs, HTTP, and REST

Objective: Understand API styles and how web clients exchange structured data.

What to teach

- Library APIs, OS APIs, and web APIs.
- HTTP request/response structure, headers, URLs, and bodies.
- GET, POST, PUT, PATCH, and DELETE.
- JSON encoding/decoding and status codes.
- REST resources, authentication basics, and API testing.

Practical demonstration

Use HTTPX to call a public test API, inspect the status code and headers, and parse the JSON response.

Student exercise: Retrieve a sample record, extract selected fields, and handle HTTP errors and timeouts.

Resources: MDN HTTP overview; HTTPX documentation.

Session 10 — Building APIs with FastAPI

Objective: Implement and test a REST API in Python.

What to teach

- Install FastAPI and run with Uvicorn.
- Routes, path parameters, query parameters, and request bodies.
- Pydantic models and input validation.
- JSON responses, status codes, and HTTPException.
- CRUD endpoints, Swagger UI, SQLite persistence, and basic security.

Practical demonstration

Create a FastAPI application with GET /students and POST /students, then test it at /docs.

Student exercise: Implement create, read, update, and delete endpoints for student records; add validation and persistent SQLite storage if time permits.

Resources: FastAPI tutorial.

Session 11 — Programming Languages for System Programming

Objective: Compare language execution models and select appropriate tools for systems tasks.

What to teach

- Compiled vs. interpreted; high-level vs. low-level languages.
- Python source, bytecode, and the Python virtual machine.
- Memory management and garbage collection vs. manual management.
- System calls, native libraries, and language bindings.
- Python vs. C/C++, Rust, and Go: performance, safety, portability, and productivity.

Practical demonstration

Compare the implementation of a small task in Python and C, or demonstrate a safe native-library call with ctypes.

Student exercise: Choose a system-programming task and justify a language choice based on performance, memory safety, OS access, and development time.

Resources: Python ctypes documentation.

Session 12 — System-Related Functions and Automation

Objective: Use Python to inspect and interact with OS resources safely.

What to teach

- Files and directories with pathlib and os.
- Environment variables and configuration.
- Launching subprocesses and capturing output.
- Process IDs, exit codes, and process status.
- Logging, exceptions, permissions, signals, and graceful shutdown.
- System monitoring with psutil; Windows/Linux differences.

Practical demonstration

Use subprocess.run() with a list of arguments to launch a harmless child process and capture its output.

Student exercise: Build a system-information utility that reports selected OS details and runs a safe external command; handle failures and log results.

Resources: subprocess; pathlib; psutil.

Session 13 — Integration Project and Review

Objective: Combine operating-system, concurrency, networking, and API concepts into a working application.

What to teach

- Review architecture and divide work into modules.
- Collect system metrics or simulated sensor readings.
- Use threads or a thread pool where justified.
- Expose data through at least three REST endpoints.
- Validate inputs, log errors, and write basic tests.
- Present the design, implementation, limitations, and security considerations.

Practical demonstration

Build a local concurrent system-monitoring API that collects sample metrics and exposes them as JSON through FastAPI. Keep the project local and use least privilege.

Student exercise: Present a working client-server application, demonstrate at least one concurrent component and three API endpoints, and submit documentation and tests.

Resources: FastAPI tutorial; Python threading and socket documentation; pytest documentation.

Suggested project and assessment

Capstone: Concurrent System Monitoring API

Students build a local application that collects system or simulated sensor metrics, processes data concurrently where appropriate, and exposes the results through a FastAPI service.

- System information collector: CPU, memory, or process information (or simulated data).
- Concurrency layer: threads, synchronization, or a thread pool.
- API layer: at least three endpoints returning JSON.
- Quality: validation, error handling, logging, tests, and documentation.
- Client: a script that calls the API and displays selected results.

Suggested grading rubric

Criterion	Weight
Correct implementation	30%
Understanding of system concepts	25%
Networking and concurrency	20%
Error handling and testing	15%
Documentation and presentation	10%

Teaching approach

A practical balance of approximately 40% conceptual teaching and 60% demonstrations and exercises is a useful starting point. Adjust based on students' Python experience and the length of each class.

Recommended core references

Python documentation • Socket Programming HOWTO • FastAPI tutorial • MDN HTTP overview • Wireshark documentation.